

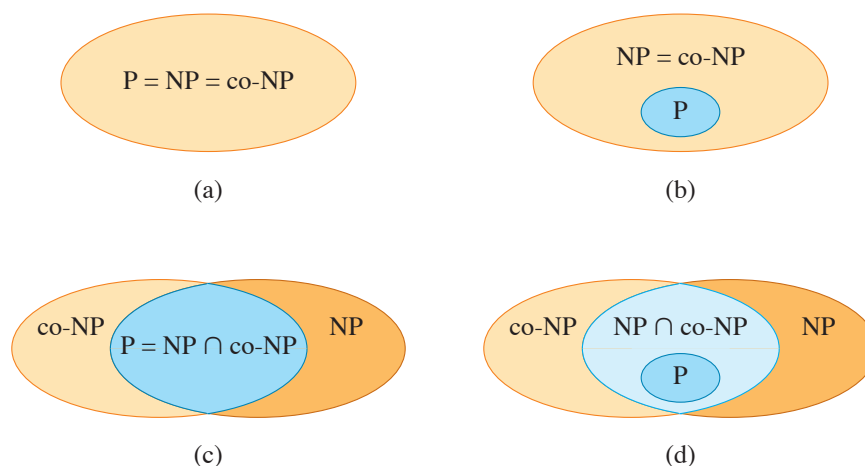


Figure 34.3 Four possibilities for relationships among complexity classes. In each diagram, one region enclosing another indicates a proper-subset relation. **(a)** $P = NP = \text{co-NP}$. Most researchers regard this possibility as the most unlikely. **(b)** If NP is closed under complement, then $NP = \text{co-NP}$, but it need not be the case that $P = NP$. **(c)** $P = NP \cap \text{co-NP}$, but NP is not closed under complement. **(d)** $NP \neq \text{co-NP}$ and $P \neq NP \cap \text{co-NP}$. Most researchers regard this possibility as the most likely.

consisting of problems for which a solution can be verified quickly. You may have learned from experience that it is often more difficult to solve a problem from scratch than to verify a clearly presented solution, especially when working under time constraints. Theoretical computer scientists generally believe that this analogy extends to the classes P and NP , and thus that NP includes languages that do not belong to P .

There is more compelling, though not conclusive, evidence that $P \neq NP$ —the existence of languages that are “NP-complete.” Section 34.3 will study this class.

Many other fundamental questions beyond the $P \neq NP$ question remain unresolved. Figure 34.3 shows some possible scenarios. Despite much work by many researchers, no one even knows whether the class NP is closed under complement. That is, does $L \in NP$ imply $\overline{L} \in NP$? We define the **complexity class co-NP** as the set of languages L such that $\overline{L} \in NP$, so that the question of whether NP is closed under complement is also whether $NP = \text{co-NP}$. Since P is closed under complement (Exercise 34.1-6), it follows from Exercise 34.2-9 ($P \subseteq \text{co-NP}$) that $P \subseteq NP \cap \text{co-NP}$. Once again, however, no one knows whether $P = NP \cap \text{co-NP}$ or whether there is some language in $(NP \cap \text{co-NP}) - P$.

Thus our understanding of the precise relationship between P and NP is woefully incomplete. Nevertheless, even though we might not be able to prove that a particular problem is intractable, if we can prove that it is NP-complete, then we have gained valuable information about it.

Exercises**34.2-1**

Consider the language $\text{GRAPH-ISOMORPHISM} = \{\langle G_1, G_2 \rangle : G_1 \text{ and } G_2 \text{ are isomorphic graphs}\}$. Prove that $\text{GRAPH-ISOMORPHISM} \in \text{NP}$ by describing a polynomial-time algorithm to verify the language.

34.2-2

Prove that if G is an undirected bipartite graph with an odd number of vertices, then G is nonhamiltonian.

34.2-3

Show that if $\text{HAM-CYCLE} \in \text{P}$, then the problem of listing the vertices of a hamiltonian cycle, in order, is polynomial-time solvable.

34.2-4

Prove that the class NP of languages is closed under union, intersection, concatenation, and Kleene star. Discuss the closure of NP under complement.

34.2-5

Show that any language in NP can be decided by an algorithm with a running time of $2^{O(n^k)}$ for some constant k .

34.2-6

A *hamiltonian path* in a graph is a simple path that visits every vertex exactly once. Show that the language $\text{HAM-PATH} = \{\langle G, u, v \rangle : \text{there is a hamiltonian path from } u \text{ to } v \text{ in graph } G\}$ belongs to NP.

34.2-7

Show that the hamiltonian-path problem from Exercise 34.2-6 can be solved in polynomial time on directed acyclic graphs. Give an efficient algorithm for the problem.

34.2-8

Let ϕ be a boolean formula constructed from the boolean input variables $x_1, x_2, \dots, x_k$, negations ($\neg$), ANDs ($\wedge$), ORs ($\vee$), and parentheses. The formula ϕ is a *tautology* if it evaluates to 1 for every assignment of 1 and 0 to the input variables. Define TAUTOLOGY as the language of boolean formulas that are tautologies. Show that $\text{TAUTOLOGY} \in \text{co-NP}$.

34.2-9

Prove that $\text{P} \subseteq \text{co-NP}$.

34.2-10

Prove that if $\text{NP} \neq \text{co-NP}$, then $\text{P} \neq \text{NP}$.

34.2-11

Let G be a connected, undirected graph with at least three vertices, and let G^3 be the graph obtained by connecting all pairs of vertices that are connected by a path in G of length at most 3. Prove that G^3 is hamiltonian. (*Hint:* Construct a spanning tree for G , and use an inductive argument.)

34.3 NP-completeness and reducibility

Perhaps the most compelling reason why theoretical computer scientists believe that $\text{P} \neq \text{NP}$ comes from the existence of the class of NP-complete problems. This class has the intriguing property that if *any* NP-complete problem can be solved in polynomial time, then *every* problem in NP has a polynomial-time solution, that is, $\text{P} = \text{NP}$. Despite decades of study, though, no polynomial-time algorithm has ever been discovered for any NP-complete problem.

The language HAM-CYCLE is one NP-complete problem. If there were an algorithm to decide HAM-CYCLE in polynomial time, then every problem in NP could be solved in polynomial time. The NP-complete languages are, in a sense, the “hardest” languages in NP. In fact, if $\text{NP} = \text{P}$ turns out to be nonempty, we will be able to say with certainty that $\text{HAM-CYCLE} \in \text{NP} = \text{P}$.

This section starts by showing how to compare the relative “hardness” of languages using a precise notion called “polynomial-time reducibility.” It then formally defines the NP-complete languages, finishing by sketching a proof that one such language, called CIRCUIT-SAT, is NP-complete. Sections 34.4 and 34.5 will use the notion of reducibility to show that many other problems are NP-complete.

Reducibility

One way that sometimes works for solving a problem is to recast it as a different problem. We call that strategy “reducing” one problem to another. Think of a problem Q as being reducible to another problem Q' if any instance of Q can be recast as an instance of Q' , and the solution to the instance of Q' provides a solution to the instance of Q . For example, the problem of solving linear equations in an indeterminate x reduces to the problem of solving quadratic equations. Given a linear-equation instance $ax + b = 0$ (with solution $x = -b/a$), you can transform it to the quadratic equation $ax^2 + bx + 0 = 0$. This quadratic equation has the solutions $x = (-b \pm \sqrt{b^2 - 4ac})/2a$, where $c = 0$, so that $\sqrt{b^2 - 4ac} = b$. The

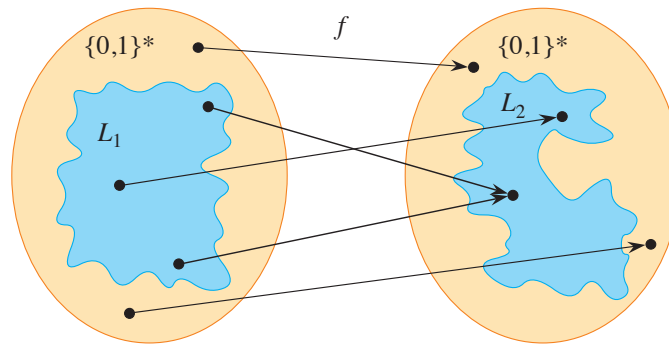


Figure 34.4 A function f that reduces language L_1 to language L_2 . For any input $x \in \{0, 1\}^*$, the question of whether $x \in L_1$ has the same answer as the question of whether $f(x) \in L_2$.

solutions are then $x = (-b + b)/2a = 0$ and $x = (-b - b)/2a = -b/a$, thereby providing a solution to $ax + b = 0$. Thus, if a problem Q reduces to another problem Q' , then Q is, in a sense, “no harder to solve” than Q' .

Returning to our formal-language framework for decision problems, we say that a language L_1 is **polynomial-time reducible** to a language L_2 , written $L_1 \leq_P L_2$, if there exists a polynomial-time computable function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ such that for all $x \in \{0, 1\}^*$,

$$x \in L_1 \text{ if and only if } f(x) \in L_2. \quad (34.1)$$

We call the function f the **reduction function**, and a polynomial-time algorithm F that computes f is a **reduction algorithm**.

Figure 34.4 illustrates the idea of a reduction from a language L_1 to another language L_2 . Each language is a subset of $\{0, 1\}^*$. The reduction function f provides a mapping such that if $x \in L_1$, then $f(x) \in L_2$. Moreover, if $x \notin L_1$, then $f(x) \notin L_2$. Thus, the reduction function maps any instance x of the decision problem represented by the language L_1 to an instance $f(x)$ of the problem represented by L_2 . Providing an answer to whether $f(x) \in L_2$ directly provides the answer to whether $x \in L_1$. If, in addition, f can be computed in polynomial time, it is a polynomial-time reduction function.

Polynomial-time reductions give us a powerful tool for proving that various languages belong to P.

Lemma 34.3

If $L_1, L_2 \subseteq \{0, 1\}^*$ are languages such that $L_1 \leq_P L_2$, then $L_2 \in P$ implies $L_1 \in P$.

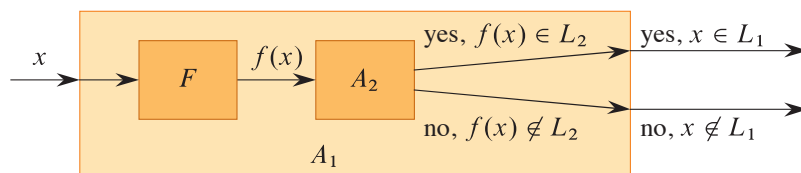


Figure 34.5 The proof of Lemma 34.3. The algorithm F is a reduction algorithm that computes the reduction function f from L_1 to L_2 in polynomial time, and A_2 is a polynomial-time algorithm that decides L_2 . Algorithm A_1 decides whether $x \in L_1$ by using F to transform any input x into $f(x)$ and then using A_2 to decide whether $f(x) \in L_2$.

Proof Let A_2 be a polynomial-time algorithm that decides L_2 , and let F be a polynomial-time reduction algorithm that computes the reduction function f . We show how to construct a polynomial-time algorithm A_1 that decides L_1 .

Figure 34.5 illustrates how we construct A_1 . For a given input $x \in \{0, 1\}^*$, algorithm A_1 uses F to transform x into $f(x)$, and then it uses A_2 to test whether $f(x) \in L_2$. Algorithm A_1 takes the output from algorithm A_2 and produces that answer as its own output.

The correctness of A_1 follows from condition (34.1). The algorithm runs in polynomial time, since both F and A_2 run in polynomial time (see Exercise 34.1-5). ■

NP-completeness

Polynomial-time reductions allow us to formally show that one problem is at least as hard as another, to within a polynomial-time factor. That is, if $L_1 \leq_P L_2$, then L_1 is not more than a polynomial factor harder than L_2 , which is why the “less than or equal to” notation for reduction is mnemonic. We can now define the set of NP-complete languages, which are the hardest problems in NP.

A language $L \subseteq \{0, 1\}^*$ is **NP-complete** if

1. $L \in \text{NP}$, and
2. $L' \leq_P L$ for every $L' \in \text{NP}$.

If a language L satisfies property 2, but not necessarily property 1, we say that L is **NP-hard**. We also define NPC to be the class of NP-complete languages.

As the following theorem shows, NP-completeness is at the crux of deciding whether P is in fact equal to NP.

Theorem 34.4

If any NP-complete problem is polynomial-time solvable, then $P = \text{NP}$. Equivalently, if any problem in NP is not polynomial-time solvable, then no NP-complete problem is polynomial-time solvable.

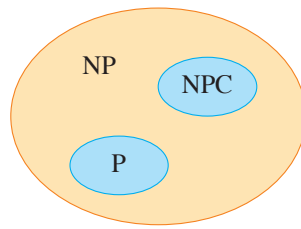


Figure 34.6 How most theoretical computer scientists view the relationships among P , NP , and NPC . Both P and NPC are wholly contained within NP , and $P \cap NPC = \emptyset$.

Proof Suppose that $L \in P$ and also that $L \in NPC$. For any $L' \in NP$, we have $L' \leq_P L$ by property 2 of the definition of NP-completeness. Thus, by Lemma 34.3, we also have that $L' \in P$, which proves the first statement of the theorem.

To prove the second statement, consider the contrapositive of the first statement: if $P \neq NP$, then there does not exist an NP-complete problem that is polynomial-time solvable. But $P \neq NP$ means that there is some problem in NP that is not polynomial-time solvable, and hence the second statement is the contrapositive of the first statement. ■

It is for this reason that research into the $P \neq NP$ question centers around the NP-complete problems. Most theoretical computer scientists believe that $P \neq NP$, which leads to the relationships among P , NP , and NPC shown in Figure 34.6. For all we know, however, someone may yet come up with a polynomial-time algorithm for an NP-complete problem, thus proving that $P = NP$. Nevertheless, since no polynomial-time algorithm for any NP-complete problem has yet been discovered, a proof that a problem is NP-complete provides excellent evidence that it is intractable.

Circuit satisfiability

We have defined the notion of an NP-complete problem, but up to this point, we have not actually proved that any problem is NP-complete. Once we prove that at least one problem is NP-complete, polynomial-time reducibility becomes a tool to prove other problems to be NP-complete. Thus, we now focus on demonstrating the existence of an NP-complete problem: the circuit-satisfiability problem.

Unfortunately, the formal proof that the circuit-satisfiability problem is NP-complete requires technical detail beyond the scope of this text. Instead, we'll informally describe a proof that relies on a basic understanding of boolean combinatorial circuits.

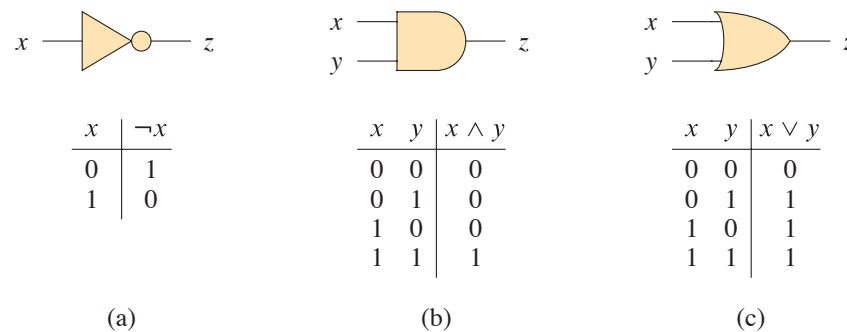


Figure 34.7 Three basic logic gates, with binary inputs and outputs. Under each gate is the truth table that describes the gate's operation. **(a)** The NOT gate. **(b)** The AND gate. **(c)** The OR gate.

Boolean combinational circuits are built from boolean combinational elements that are interconnected by wires. A *boolean combinational element* is any circuit element that has a constant number of boolean inputs and outputs and that performs a well-defined function. Boolean values are drawn from the set $\{0, 1\}$, where 0 represents FALSE and 1 represents TRUE.

The boolean combinational elements appearing in the circuit-satisfiability problem compute simple boolean functions, and they are known as *logic gates*. Figure 34.7 shows the three basic logic gates used in the circuit-satisfiability problem: the *NOT gate* (or *inverter*), the *AND gate*, and the *OR gate*. The NOT gate takes a single binary *input* x , whose value is either 0 or 1, and produces a binary *output* z whose value is opposite that of the input value. Each of the other two gates takes two binary inputs x and y and produces a single binary output z .

The operation of each gate, or of any boolean combinational element, is defined by a *truth table*, shown under each gate in Figure 34.7. A truth table gives the outputs of the combinational element for each possible setting of the inputs. For example, the truth table for the OR gate says that when the inputs are $x = 0$ and $y = 1$, the output value is $z = 1$. The symbol $\neg$ denotes the NOT function, $\wedge$ denotes the AND function, and $\vee$ denotes the OR function. Thus, for example, $0 \vee 1 = 1$.

AND and OR gates are not limited to just two inputs. An AND gate's output is 1 if all of its inputs are 1, and its output is 0 otherwise. An OR gate's output is 1 if any of its inputs are 1, and its output is 0 otherwise.

A *boolean combinational circuit* consists of one or more boolean combinational elements interconnected by *wires*. A wire can connect the output of one element to the input of another, so that the output value of the first element becomes an input value of the second. Figure 34.8 shows two similar boolean combinational circuits, differing in only one gate. Part (a) of the figure also shows the values on

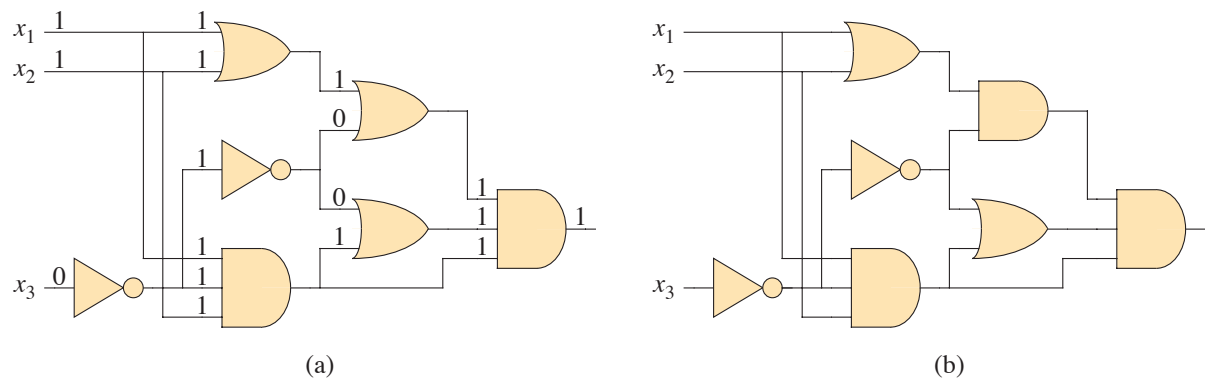


Figure 34.8 Two instances of the circuit-satisfiability problem. (a) The assignment $\langle x_1 = 1, x_2 = 1, x_3 = 0 \rangle$ to the inputs of this circuit causes the output of the circuit to be 1. The circuit is therefore satisfiable. (b) No assignment to the inputs of this circuit can cause the output of the circuit to be 1. The circuit is therefore unsatisfiable.

the individual wires, given the input $\langle x_1 = 1, x_2 = 1, x_3 = 0 \rangle$. Although a single wire may have no more than one combinational-element output connected to it, it can feed several element inputs. The number of element inputs fed by a wire is called the *fan-out* of the wire. If no element output is connected to a wire, the wire is a *circuit input*, accepting input values from an external source. If no element input is connected to a wire, the wire is a *circuit output*, providing the results of the circuit's computation to the outside world. (An internal wire can also fan out to a circuit output.) For the purpose of defining the circuit-satisfiability problem, we limit the number of circuit outputs to 1, though in actual hardware design, a boolean combinational circuit may have multiple outputs.

Boolean combinational circuits contain no cycles. In other words, for a given combinational circuit, imagine a directed graph $G = (V, E)$ with one vertex for each combinational element and with k directed edges for each wire whose fan-out is k , where the graph contains a directed edge (u, v) if a wire connects the output of element u to an input of element v . Then G must be acyclic.

A *truth assignment* for a boolean combinational circuit is a set of boolean input values. We say that a 1-output boolean combinational circuit is *satisfiable* if it has a *satisfying assignment*: a truth assignment that causes the output of the circuit to be 1. For example, the circuit in Figure 34.8(a) has the satisfying assignment $\langle x_1 = 1, x_2 = 1, x_3 = 0 \rangle$, and so it is satisfiable. As Exercise 34.3-1 asks you to show, no assignment of values to x_1 , x_2 , and x_3 causes the circuit in Figure 34.8(b) to produce a 1 output. Since it always produces 0, it is unsatisfiable.

The *circuit-satisfiability problem* is, “Given a boolean combinational circuit composed of AND, OR, and NOT gates, is it satisfiable?” In order to pose this

question formally, however, we must agree on a standard encoding for circuits. The *size* of a boolean combinational circuit is the number of boolean combinational elements plus the number of wires in the circuit. We could devise a graph-like encoding that maps any given circuit C into a binary string $\langle C \rangle$ whose length is polynomial in the size of the circuit itself. As a formal language, we can therefore define

$$\text{CIRCUIT-SAT} = \{ \langle C \rangle : C \text{ is a satisfiable boolean combinational circuit} \}.$$

The circuit-satisfiability problem arises in the area of computer-aided hardware optimization. If a subcircuit always produces 0, that subcircuit is unnecessary: the designer can replace it by a simpler subcircuit that omits all logic gates and provides the constant 0 value as its output. You can see the value in having a polynomial-time algorithm for this problem.

Given a circuit C , you can determine whether it is satisfiable by simply checking all possible assignments to the inputs. Unfortunately, if the circuit has k inputs, then you would have to check up to 2^k possible assignments. When the size of C is polynomial in k , checking all possible assignments to the inputs takes $\Omega(2^k)$ time, which is superpolynomial in the size of the circuit.¹⁰ In fact, as we have claimed, there is strong evidence that no polynomial-time algorithm exists that solves the circuit-satisfiability problem because circuit satisfiability is NP-complete. We break the proof of this fact into two parts, based on the two parts of the definition of NP-completeness.

Lemma 34.5

The circuit-satisfiability problem belongs to the class NP.

Proof We provide a two-input, polynomial-time algorithm A that can verify CIRCUIT-SAT. One of the inputs to A is (a standard encoding of) a boolean combinational circuit C . The other input is a certificate corresponding to an assignment of a boolean value to each of the wires in C . (See Exercise 34.3-4 for a smaller certificate.)

The algorithm A works as follows. For each logic gate in the circuit, it checks that the value provided by the certificate on the output wire is correctly computed as a function of the values on the input wires. Then, if the output of the entire circuit is 1, algorithm A outputs 1, since the values assigned to the inputs of C provide a satisfying assignment. Otherwise, A outputs 0.

¹⁰ On the other hand, if the size of the circuit C is $\Theta(2^k)$, then an algorithm whose running time is $O(2^k)$ has a running time that is polynomial in the circuit size. Even if $P \neq NP$, this situation would not contradict the NP-completeness of the problem. The existence of a polynomial-time algorithm for a special case does not imply that there is a polynomial-time algorithm for all cases.

Whenever a satisfiable circuit C is input to algorithm A , there exists a certificate whose length is polynomial in the size of C and that causes A to output a 1. Whenever an unsatisfiable circuit is input, no certificate can fool A into believing that the circuit is satisfiable. Algorithm A runs in polynomial time, and with a good implementation, linear time suffices. Thus, CIRCUIT-SAT is verifiable in polynomial time, and CIRCUIT-SAT $\in$ NP. ■

The second part of proving that CIRCUIT-SAT is NP-complete is to show that the language is NP-hard: that *every* language in NP is polynomial-time reducible to CIRCUIT-SAT. The actual proof of this fact is full of technical intricacies, and so instead we'll sketch the proof based on some understanding of the workings of computer hardware.

A computer program is stored in the computer's memory as a sequence of instructions. A typical instruction encodes an operation to be performed, addresses of operands in memory, and an address where the result is to be stored. A special memory location, called the *program counter*, keeps track of which instruction is to be executed next. The program counter automatically increments when each instruction is fetched, thereby causing the computer to execute instructions sequentially. Certain instructions can cause a value to be written to the program counter, however, which alters the normal sequential execution and allows the computer to loop and perform conditional branches.

At any point while a program executes, the computer's memory holds the entire state of the computation. (Consider the memory to include the program itself, the program counter, working storage, and any of the various bits of state that a computer maintains for bookkeeping.) We call any particular state of computer memory a *configuration*. When an instruction executes, it transforms the configuration. Think of an instruction as mapping one configuration to another. The computer hardware that accomplishes this mapping can be implemented as a boolean combinational circuit, which we denote by M in the proof of the following lemma.

Lemma 34.6

The circuit-satisfiability problem is NP-hard.

Proof Let L be any language in NP. We'll describe a polynomial-time algorithm F computing a reduction function f that maps every binary string x to a circuit $C = f(x)$ such that $x \in L$ if and only if $C \in \text{CIRCUIT-SAT}$.

Since $L \in \text{NP}$, there must exist an algorithm A that verifies L in polynomial time. The algorithm F that we construct uses the two-input algorithm A to compute the reduction function f .

Let $T(n)$ denote the worst-case running time of algorithm A on length- n input strings, and let $k \geq 1$ be a constant such that $T(n) = O(n^k)$ and the length of the